



Sección Crítica



Una **sección crítica** es la parte del código donde un proceso/hilo accede a un **recurso compartido** que **no puede usarse simultáneamente** por más de una entidad concurrente.

Explicación

La idea es muy sencilla:

Es un fragmento de instrucciones que deben ejecutarse como si fueran atómicas, porque operan sobre datos que serían inconsistentes si dos hilos los modifican a la vez.

Cuando un hilo entra a la sección crítica, **ningún otro puede hacerlo** hasta que el primero termine.

Los hilos que intenten entrar mientras está ocupada deberán esperar.

Una sección crítica tiene un protocolo de entrada y otro de salida.

```
while (true) {  
    // 1. Sección no crítica  
    hacer_otra_cosa();  
  
    // 2. Protocolo de entrada  
    entrar_seccion_critica();  
  
    // 3. Sección crítica  
    // (acceso a recurso compartido)  
    recurso_compartido = recurso_compartido + 1;  
}
```

```
// 4. Protocolo de salida
salir_seccion_critica();
}
```

¿Qué se exige a una sección crítica?

Por su naturaleza, podríamos decir que si una sección crítica está bien diseñada en un mismo instante de tiempo t NO PUEDE HABER más de un hilo ahí.

Una solución a la exclusión mutua debe garantizar tres propiedades fundamentales:

1) Exclusión mutua

Solo **un** hilo puede estar en la sección crítica en un instante dado.

2) Progreso

Si la sección crítica está libre y hay hilos que quieren entrar,

alguien debe entrar en un tiempo finito.

No puede depender eternamente de decisiones de hilos que no quieren usar la sección crítica.

3) Espera limitada (bounded waiting)

Si un proceso quiere entrar a su sección crítica,

no puede esperar indefinidamente mientras otros entran repetidamente antes que él.

Debe existir un límite al número de veces que otros pueden adelantarse.

Estas propiedades son las que se usan para demostrar corrección de protocolos de entrada y salida.

Ejemplo seguro

```
var mutex := 1;  -- semáforo binario

process Hilo:
  while true do
    hacer_otra_cosa();
```

```
wait(mutex);    -- protocolo de entrada
recurso := recurso + 1; -- sección crítica
signal(mutex);  -- protocolo de salida
end while
```

Ejemplo inseguro

```
recurso := recurso + 1;
```

Esto que parece simple se convierte en tres instrucciones máquina:

1. cargar recurso en un registro
2. sumar 1
3. almacenar el resultado

Si dos hilos intercalan estas instrucciones, el resultado **no es determinista** → condición de carrera.

Partes de la sección crítica

Protocolo de Entrada

Es el código que decide si el hilo puede o no entrar en la sección crítica.

- Puede haber muchos hilos ejecutando el protocolo de entrada simultáneamente en un mismo instante t.
- Solo uno podrá cruzar a la sección crítica en un instante dado.

Ejemplos conceptuales:

`wait(mutex)` , `lock()` , comprobaciones de turnos (Dekker/Peterson), etc.

Sección Crítica

El bloque que accede realmente al recurso compartido.

Ejemplos típicos:

- modificar un contador,

- insertar en un buffer compartido,
- acceder a una variable no atómica,
- imprimir por pantalla (porque `printf` no es atómico).

En un instante dado, **solo un hilo puede estar aquí**.

Muy importante asegurarnos que las instrucciones que utilizamos en esta sección son atómicas o son *thread-safe*, ya que podríamos utilizar métodos que no lo sean y generar inconsistencias (por ejemplo si llamamos a otra función o usamos funciones de bibliotecas externas).

Por ejemplo:

```
semaphore mutex := 1  -- para proteger 'contador'
var contador := 0     -- recurso compartido

procedure log(mensaje):
  -- Esta función NO es thread-safe.
  -- Internamente escribe en un fichero compartido
  -- sin ningún tipo de exclusión mutua.
  fichero_compartido.write(mensaje)

process Trabajador:
  while true do
    wait(mutex)          -- protocolo de entrada

    -- SECCIÓN CRÍTICA (sobre 'contador')
    contador := contador + 1

    -- ⚠ Peligro: llamamos a una función que NO es thread-safe
    log("contador vale " + toString(contador))

    signal(mutex)        -- protocolo de salida

    hacer_otra_cosa()
  end while
```

Qué pasa aquí:

- El acceso a `contador` está protegido.
- Pero `log()` puede ser llamada **en paralelo** por otros procesos/hilos.
- Resultado:
 - líneas mezcladas en el fichero,
 - mensajes incoherentes,
 - incluso corrupción interna si la librería de escritura no es thread-safe.

Has protegido *una parte* del acceso, pero dentro de tu SC has metido código que toca **otro recurso compartido** sin protección → inconsistencia.

Protocolo de Salida

Código que indica que la sección crítica queda libre.

Ejemplos:

- `signal(mutex)`
- `unlock()`
- cambiar una variable de turno (Peterson)
- incrementar un semáforo

Debe ser **exclusivo también**: solo el hilo dentro de la sección crítica puede ejecutarlo, y nunca debe ser ejecutado por dos hilos simultáneamente (porque eso significaría que dos han salido de la SC al mismo tiempo → la SC estaba rota).

Soluciones para garantizar la EM en las secciones críticas

A lo largo de la asignatura se ven varios enfoques para garantizar la exclusión mutua en las secciones críticas:

- Algoritmos de espera ocupada.
- Soluciones de hardware.
- Semáforos.

- Cerrojos.
- Monitores.
- Regiones críticas.
- Regiones críticas condicionales.

Para saber de qué va cada uno revisar los apuntes dedicados de cada parte.